

CodeSheriff: Algorithms and Equations for Calibrated Multi-Witness Vulnerability Inference in Pull Requests

Shubhankar Patil
CodeSheriff Project
shubhankarp258@gmail.com

Abstract—CodeSheriff analyses each function changed by a pull request with four heterogeneous analysis agents and fuses their statements into a *calibrated* posterior probability that the change introduces a vulnerability. This paper states the algorithm of each agent and the fusion and calibration arithmetic in closed form, exactly as implemented. The central design commitment is that the four agents are treated as four *witnesses* rather than four detectors: fusion multiplies one likelihood ratio per witness whatever each agent said, an agent that could not run contributes the multiplicative identity, and every ratio, the prior and the alert threshold are estimated on disjoint data splits.

Index Terms—static analysis, taint analysis, large language models, software fault isolation, Bayesian inference, probability calibration, expected calibration error

I. NOTATION AND THE UNIT OF ANALYSIS

A. Change unit

A pull request is decomposed into *change units*. A change unit $u = \langle f, \sigma, s^-, s^+, \ell, D \rangle$ is one outermost changed function: f is the file path, σ the qualified symbol, s^- and s^+ the base and head source text of that function, ℓ the language and D the set of changed line numbers, derived by textual diff over the two file blobs rather than from the forge’s patch representation. A file whose change lies outside any function yields a single `<module>` unit carrying the whole file.

B. Finding key

Agents never see one another’s output, so they must nevertheless agree on *which* finding they are describing. The join is a content-free key over the unit and the weakness class, and it deliberately excludes the sink expression:

$$k(u, c) = \text{SHA256}(f \parallel :: \parallel \sigma \parallel :: \parallel c)_{[0:16]} \quad (1)$$

where c is drawn from a closed in-scope set of ten CWE classes. Including the sink text in (1) causes a taint engine reporting `cursor.execute(query)` and a language model reporting `cursor.execute` to produce two distinct keys for one bug; every finding then becomes a singleton and the inference engine performs no update at all.

C. Evidence

Every agent returns at least one *evidence* item per unit, of exactly one of three kinds:

- DETECTION $\langle a, k, c, r \rangle$: agent a claims weakness c at key k with raw score $r \in [0, 1]$;

- SILENCE $\langle a, C_a \rangle$: agent a ran to completion and found nothing, where C_a is the set of weakness classes it is capable of reporting;
- ABSTENTION $\langle a, \rho \rangle$: agent a could not run, with reason ρ .

The distinction between the last two is load-bearing and is the reason a boolean `abstained` flag is insufficient: silence is an observation and is assigned a likelihood ratio below unity, whereas abstention is the absence of an observation and is assigned exactly 1. Returning an empty list from a successful analysis is a contract violation, because it is indistinguishable from silence while carrying none of its meaning.

D. Witnesses

Let $\mathcal{W} = \{\text{STRUCTURAL}, \text{SEMANTIC}, \text{CONTEXT}, \text{RUNTIME}\}$. A *backend* is an implementation; a *witness* is a basis for a decision. The map $\omega : a \mapsto W$ is total and fixed, and an unregistered a raises rather than becoming a fifth witness. In particular `structural.taint` and `structural.semgrep` both map to STRUCTURAL: they are two rule-based analyses of the same source text and are maximally correlated, so fusing them as independent factors multiplies $8.5 \times 7.0 = 59.5$ out of a single witness.

II. AGENT I: THE STRUCTURAL WITNESS

Basis: a mechanical reachability proof over source syntax.
Failure mode: a bug with no syntactic pattern.

A. Def-use graph

The head source s^+ is parsed with tree-sitter and reduced to a directed graph $G = (N, E)$ whose nodes are *definitions* — points where a value enters, is redefined, is validated, or is used — and whose edges are flows. Every edge carries the set of sink classes it neutralises,

$$\text{clears} : E \rightarrow 2^{\mathcal{C}}, \quad \mathcal{C} = \{\text{SQL}, \text{XSS}, \text{CMD}, \text{PATH}, \dots\} \quad (2)$$

so that an `html.escape` edge clears XSS on the hop it actually wraps and nowhere else. Three modelling choices are stated explicitly, because each is aggressive and each is necessary:

- 1) **Every parameter is an origin.** The unit of analysis is one function, so its signature is the trust boundary and anything arriving through it comes from code this

Algorithm 1 STRUCTURALTAINT(u)

Input: change unit u ; source, sink and sanitiser catalogue

Output: a list of evidence items; never an exception

```
1: if  $\ell$  unsupported or  $|s^+| > 400\,000$  bytes then
2:   return ABSTENTION(unsupported/too_large)
3: end if
4:  $G \leftarrow \text{BUILDDEFUSE}(\text{PARSE}(s^+))$ 
5:  $\mathcal{K} \leftarrow \emptyset$   $\triangleright$  key  $\rightarrow$  strongest evidence
6: for all sink nodes  $s$  with rule  $r$  whose CWE is in scope
  do
7:    $c \leftarrow \text{class}(r)$ 
8:   if  $T_c$  uncomputed then  $T_c \leftarrow \text{fixpoint (4) on } G_c$ 
9:    $o \leftarrow \text{origin}_{T_c}(s)$ ; if  $o = \perp$  then continue
10:   $\pi \leftarrow \text{shortestPath}_{G_c}(o, s)$ 
11:  if (5) fails then continue
12:   $\kappa \leftarrow k(u, \text{cwe}(r))$ ;  $\varrho \leftarrow (6)$  with  $L = |\pi|$ 
13:  keep in  $\mathcal{K}[\kappa]$  whichever of the old and new evidence
    has larger  $\varrho$ 
14: end for
15: return  $\mathcal{K}$ , plus SILENCE( $C_a \setminus \text{reported CWEs}$ )
```

analysis cannot see. Formally $O = \{n \in N : \text{role}(n) = \text{SOURCE}\}$ contains every non-receiver parameter.

- 2) **A validating guard is a definition.** `if x not in ALLOWED: raise` creates a node redefining x with an incoming edge clearing all classes, so every later use resolves through it. An emptiness test such as `if x is None: return` establishes nothing about the value and creates no such node.
- 3) **Path sinks require composition.** A sink rule may set $\text{req}(r) = \text{true}$, in which case a whole path handed in and used unchanged is the caller's choice, not a traversal.

B. Propagation

For sink class c define the sanitiser-quotient graph

$$G_c = (N, \{e \in E : c \notin \text{clears}(e)\}). \quad (3)$$

The tainted set for c is the least fixpoint of the transfer relation seeded at the origins,

$$T_c = \mu X. O \cup \{n' \in N : \exists n \in X, (n, n') \in E(G_c)\}, \quad (4)$$

computed by the worklist of Algorithm 1, which additionally records for each tainted node the origin its taint came from, so that a reported path can be rendered back to where the value entered rather than merely asserted to exist.

A sink node s carrying rule r at argument position j is reported iff

$$s \in T_{c(r)} \wedge \pi \neq \emptyset \wedge (\neg \text{req}(r) \vee \text{comp}(\pi)), \quad (5)$$

where $\pi = \text{shortestPath}_{G_{c(r)}}(o, s)$ and $\text{comp}(\pi)$ holds iff some node on π joins a base onto the tainted value.

C. Score

The raw score is deterministic and explicit. With danger class d , path length $L = |\pi|$, partial-sanitiser flag ς , network-facing flag ν and test-file flag τ ,

$$\varrho_{\text{str}} = \text{clamp}_{[0,1]} \left(0.45 + \delta(d) - 0.25 \varsigma \right. \\ \left. - 0.0125 \min(\max(0, L - 2), 4) \right. \\ \left. + 0.15 \nu - 0.20 \tau \right) \quad (6)$$

with $\delta(\text{CRITICAL}) = 0.20$, $\delta(\text{HIGH}) = 0.10$ and $\delta(\cdot) = 0$ otherwise. Equation (6) explains an empirical observation reported in Section VIII: with $\nu = 1$ and $\varsigma = 0$, a critical sink at path length $L \geq 3$ scores at most 0.7875, so nearly every true structural detection falls in the *medium* tier and the *high* tier is populated by a single case.

III. AGENT II: THE SEMANTIC WITNESS

Basis: absorbed model knowledge — intent, then trust boundaries, then the violated invariant. *Failure mode:* the model is confidently wrong, or agreeable.

A. Prompt construction and the injection boundary

The analysed source is attacker-controlled text. It is enclosed by a *per-request random sentinel* drawn from a cryptographic source, and no other delimiter appears in the prompt. The anti-sycophancy exemplars are placed *inside* the same sentinel as the unit: they are the longest and most attended-to part of the prompt, and framing them differently would teach the model that the sentinel is decorative formatting, which is precisely the belief an injection needs.

B. Self-consistency

The model is sampled $n = 3$ times at temperature 0.3 with varying seeds, yielding responses $R_1, \dots, R_n$. Findings are clustered by key rather than by the model's own wording for the sink:

$$\mathcal{G}_\kappa = \left\{ \phi \in \bigcup_i R_i : k(u, \text{cwe}(\phi)) = \kappa \right\}, \quad (7)$$

and the raw score of a surviving cluster is the agreement rate

$$\varrho_{\text{sem}}(\kappa) = \frac{|\mathcal{G}_\kappa|}{n}. \quad (8)$$

Because (8) measures the *spread* between independently seeded samples, replaying one cached answer n times would present itself as unanimous confidence; the sampling is therefore genuinely n -fold, or the score is meaningless. Cluster attributes merge by max over severity and exploitability and by union over evidence lines and named untrusted sources.

C. Hallucination gate

Let ϕ be a candidate finding and L_u the line count of s^+ . The gate is the conjunction

$$\Gamma(\phi, u) = [\text{cwe}(\phi) \in \text{CWE}_{\text{scope}}] \wedge [\text{file}(\phi) = f] \\ \wedge [\text{sink}(\phi) \sqsubseteq s^+] \\ \wedge [\forall \lambda \in \text{lines}(\phi) : 1 \leq \varphi(\lambda) \leq L_u] \quad (9)$$

Algorithm 2 SEMANTICHOSTED(u)

Input: change unit u ; exemplar set; per-unit budget B

```
1:  $\varsigma \leftarrow \text{RANDOMSENTINEL}()$ ;  $P \leftarrow \text{BUILDPROMPT}(u, \varsigma)$ 
2:  $\mathcal{R} \leftarrow \emptyset$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:   if  $\text{spend} \geq B$  then break
5:    $\mathcal{R} \leftarrow \mathcal{R} \cup \{\text{SAMPLE}(P, \text{seed}_i, T=0.3)\}$ 
6: end for
7: if  $\mathcal{R} = \emptyset$  then return ABSTENTION(budget)
8:  $\Phi \leftarrow \{\phi \in \mathcal{R} : \Gamma(\phi, u)\} \quad \triangleright \text{gate (9)}$ 
9: cluster  $\Phi$  by  $k(u, \cdot)$ ; score each cluster by (8)
10: screen prose; emit one DETECTION per cluster
11: return detections, or SILENCE( $C_a$ ) if none survived
```

where $\sqsubseteq$ denotes verbatim substring containment and φ normalises absolute line numbers to unit-relative ones. A rejection under (9) discards that finding and *not* the whole sample, because one response can carry a real finding and an invention side by side. Comparison on the exact file path rather than the basename is required: matching on basename accepted a finding about `app/models/user.py` against `tests/user.py`. There is no tolerance term on the line bound, since a slack of +5 accepts lines that do not exist, which is the hallucination the gate exists for.

D. Prose screening

Model prose is screened at the single point where a finding becomes evidence. Text that reads as an instruction to whoever sees it next is *dropped*, not escaped, and the finding is republished from its validated fields alone. Escaping preserves the information and fixes the rendering, but an escaped injection is still delivered to the reader it targets; whether the code is vulnerable does not depend on how the model chose to describe it.

A budget exhausted before the first sample must yield ABSTENTION and never SILENCE: the latter asserts “reviewed the unit and found nothing” across all ten in-scope classes, at a likelihood ratio below 1, having read nothing.

IV. AGENT III: THE CONTEXT WITNESS

Basis: this repository’s own merged precedent. *Failure mode:* a repository with no relevant history. There is deliberately no language model in this agent; a model-driven variant would share the semantic witness’s failure mode, and heterogeneity is the entire justification for fusing four witnesses.

A. Retrieval

Let $\mathcal{P}(u)$ be the top- k ($k = 5$) merged code excerpts retrieved from the vector precedent store for repository ϱ , restricted to cosine similarity $\geq \tau_{\min} = 0.35$. Repository scoping is structural: ϱ is bound when the retriever is constructed and the retrieval call takes no repository argument, so cross-repository retrieval is unrepresentable rather than merely forbidden. Precedent is written by an offline backfill, one

chunk per symbol and only the merged side, so a control that a pull request deliberately removed cannot go on establishing itself forever.

B. Establishment

Let $\Sigma(s)$ be the *control surface* of source s — the decorators and guard calls it applies. For a control γ let $\text{car}(\gamma) = \{p \in \mathcal{P} : \gamma \in \Sigma(s_p)\}$, and let $\text{sym}(\cdot)$ denote qualified symbols. The repository *establishes* γ iff

$$\underbrace{\exists p \in \text{car}(\gamma) : \text{sym}(p) = \sigma}_{\text{the same symbol carried it}} \vee \underbrace{|\{\text{sym}(p)\}_{p \in \text{car}(\gamma)}| \geq 2}_{\text{sibling convention}} \quad (10)$$

Matching is on the symbol and never on the path, because a move is exactly what changes the path. The sibling threshold is 2 and not 1: every function calls something no other function happens to call, so treating a single neighbour’s habit as a rule would make every added function a regression against whichever neighbour retrieval returned.

C. Missing controls

With $A = \Sigma(s^+)$ the controls the unit applies, and a *fixed*, *partial* map $\text{cwe} : \Gamma \rightarrow \{\text{CWE-862}, \text{CWE-639}\}$,

$$M = \{\gamma \in E \setminus A : \text{name}(\gamma) \notin \text{names}(A) \wedge \text{cwe}(\gamma) \downarrow\}. \quad (11)$$

Which controls a repository applies is *learned*; which of them are authorisation controls is *fixed*. Making both halves learned would infer a weakness class from a name whose meaning the agent was never told; making both halves fixed would be a second rule engine with a hard-coded guard list, correlated with the one that already exists. A mined control mapping to no in-scope class therefore cannot be reported at all — escaping is not authorisation, so a repository where every merged view calls `escape()` has a real convention, a unit that drops it has really regressed, and this witness still says nothing.

D. Score

For $m \in M$ with support $\text{sup}(m)$ (distinct supporting symbols) and mean supporting similarity $\bar{s}(m)$,

$$\varrho_{\text{ctx}}(m) = b(m) \cdot \min\left(1, \frac{\bar{s}(m)}{0.85}\right), \quad b(m) = \begin{cases} 0.85 & \text{strong} \\ 0.65 & \text{weak} \end{cases} \quad (12)$$

where *strong* means the same symbol carried the control or $\text{sup}(m) \geq 3$. Damping is one-sided by construction: multiplying by raw similarity instead demoted a strong claim backed by excerpts at a perfectly good 0.9 to 0.765, so retrieval doing its job pushed every detection out of the high tier and the tier boundary ceased to mean anything.

A store that is down and a repository that is new are different facts and receive different reasons; conflating them is what let a degenerate term-hashing fallback report noise with no log line.

V. AGENT IV: THE RUNTIME WITNESS

Basis: direct observation. *Failure mode:* code that will not run in a sandbox. It observes; it never proves.

Algorithm 3 CONTEXTRAG(u)**Input:** change unit u ; repository-bound retriever

```

1: try  $\mathcal{P} \leftarrow \text{RETRIEVE}(u, k)$  catch unavailable
2:   return ABSTENTION(retrieval_unavailable)
3: if  $\mathcal{P} = \emptyset$  then return ABSTENTION(no_precedent)
4:  $E \leftarrow$  controls satisfying (10);  $A \leftarrow \Sigma(s^+)$ 
5:  $M \leftarrow$  established, unapplied, in-scope controls
6: for all CWE  $c$  with  $M_c \neq \emptyset$  do
7:   emit DETECTION at  $k(u, c)$  with score  $\max_{m \in M_c} (12)$ 
8: end for
9: return detections, or SILENCE( $\{862, 639\}$ ) if  $M = \emptyset$ 

```

A. The sandbox

The changed function is executed *once* inside a WebAssembly (Wasmtime + WASI) sandbox under policy

$$\Pi = \langle \text{fuel } 2 \times 10^{10}, \text{ wall clock } 15\text{s}, \text{ memory } 512\text{MiB} \rangle.$$

Capabilities are *absent*, not filtered: the guest environment is empty because nothing is inherited, not because a deny-list removed the interesting keys, and WASI preview1 defines no `sock_connect`, so a module requesting network access fails to instantiate — denial before the first guest instruction. The caps cover one another’s blind spots: fuel is deterministic and is what a reproducible calibration run requires, epoch interruption is the wall-clock backstop for a guest blocked inside a host call, and the memory cap keeps an allocation loop a guest-side `MemoryError` rather than a host OOM. The interpreter module is pinned by digest and verified on load; a mismatch is fatal, and its absence is an abstention rather than a fallback to host execution.

B. Taint as substring

Each parameter x_i is bound to a probe value whose text is a unique random token t_i , implemented as a `str` subclass. Taint is then a string relation rather than a wrapper discipline:

$$\text{tainted}(v) \iff \exists i : t_i \sqsubseteq \text{str}(v), \quad (13)$$

so `f-strings`, `+`, `%`, `.format()` and `os.path.join` propagate taint for free, because they propagate the characters. Composition — the runtime analogue of the structural composition rule — is

$$\text{comp}(v) \iff \neg(|\mathcal{T}(v)| = 1 \wedge \text{str}(v) = t), \quad \{t\} = \mathcal{T}(v), \quad (14)$$

that is, the value is not merely the token passed through unchanged.

C. Observation and soundness

An observation $o = \langle \text{sink}, j, \text{comp}, \text{grd} \rangle$ is recorded when a tokenised value occupies dangerous argument position j of a catalogued sink, subject to that sink’s own composition requirement. The run is partitioned by

$$\mathcal{O}_{\text{snd}} = \{o : \neg \text{grd}(o)\}, \quad \mathcal{O}_{\text{grd}} = \{o : \text{grd}(o)\}, \quad (15)$$

where $\text{grd}(o)$ holds if the token had already been passed to a call the probe could not evaluate. The output rule is

$$\text{OUT} = \begin{cases} \text{DETECTION per CWE} & \mathcal{O}_{\text{snd}} \neq \emptyset \\ \text{ABSTENTION(guard)} & \mathcal{O}_{\text{snd}} = \emptyset, \mathcal{O}_{\text{grd}} \neq \emptyset \\ \text{SILENCE}(C_a) & \text{otherwise.} \end{cases} \quad (16)$$

The middle case of (16) is the substantive one. The value demonstrably reached the sink, but the run reached it only because the harness answered a guard it could not evaluate — which may well have been the validator that makes the code safe. Reporting is a false positive on exactly the code that defends itself; staying silent argues, at a likelihood ratio below 1, that guarded code is clean on the strength of a guard nobody read. Neither is honest, so the witness states that it could not tell.

D. Score and scope

An execution either happened or it did not, so there is one tier:

$$\varrho_{\text{rt}} = 0.85. \quad (17)$$

The witness reports five CWE classes, not ten. SQL injection and cross-site scripting are excluded because their sinks are methods on objects the probe itself fabricated, so an “observation” there would be the harness observing itself — a weaker restatement of the structural witness, correlated with it. Published prose is assembled entirely from the project’s own sink table and verbs, so no substring of it originates in the analysed code or the guest trace, and the CWE is re-derived host-side from that table: analysed code can neither invent a finding nor relabel one.

VI. EVIDENCE AGGREGATION AND BAYESIAN FUSION

A. Cell selection

Let $\mathcal{Z} = \{H, M, L, S\}$ (detection-high, detection-medium, detection-low, silence). Detection tiers are

$$\text{tier}(r) = \begin{cases} H & r \geq 0.8 \\ M & 0.5 \leq r < 0.8 \\ L & r < 0.5. \end{cases} \quad (18)$$

For witness W and finding key κ with weakness c , let D_W be W ’s detections at κ and S_W its silences about the unit. The cell is

$$\text{cell}(W, \kappa) = \begin{cases} \text{tier}(\max_{e \in D_W} r(e)) & D_W \neq \emptyset \\ S & D_W = \emptyset \wedge \exists e \in S_W : c \in C_{a(e)} \\ \perp & \text{otherwise.} \end{cases} \quad (19)$$

Backends behind one witness combine by plain max: two structural backends both firing high remain one high detection rather than compounding, and one backend’s silence does not retract its sibling’s detection. The guard $c \in C_a$ in (19) is essential — the taint engine holds no CWE-862 rules whatsoever, so counting its silence there would suppress

every authorisation finding on principle. Equation (19) is implemented *once* and read by both fusion and fitting; two implementations would fit one quantity and apply another, and both halves would pass their own tests.

B. Posterior

With prior π and fitted tables $\Lambda_W : \mathcal{Z} \rightarrow \mathbb{R}_{>0}$, the posterior odds are a product over *all four* witnesses:

$$\begin{aligned} \text{odds}_{\text{post}}(\kappa) &= \frac{\pi}{1 - \pi} \prod_{W \in \mathcal{W}} \lambda_W(\kappa), \quad (20) \\ \lambda_W(\kappa) &= \begin{cases} \text{clamp}_{[\lambda_{\min}, \lambda_{\max}]}(\Lambda_W[\text{cell}(W, \kappa)]) & \text{cell} \neq \perp \\ 1 & \text{cell} = \perp, \end{cases} \quad (21) \\ p(\kappa) &= \frac{\text{odds}_{\text{post}}(\kappa)}{1 + \text{odds}_{\text{post}}(\kappa)}, \quad \text{ALERT} \iff p(\kappa) \geq \theta^*. \quad (22) \end{aligned}$$

Four properties of (20)–(21) are deliberate, and each corrects a specific defect measured in an earlier implementation.

- 1) **The product ranges over $\mathcal{W}$, not over the agents that spoke.** Every detection tier exceeds 1, so iterating only the agents that alerted makes the posterior monotonically non-decreasing in the number of agents that alerted: an agent that looked and found nothing could not lower the number, and an agent that could not look was indistinguishable from one that had.
- 2) **One factor per witness, not per backend.** See ω in Section I-D.
- 3) **Abstention is exactly 1 and has no table entry.** An unfitted constant looks like an oversight; it is the multiplicative identity *by definition*, and a fitted number there would assert that the act of failing carried information about the code.
- 4) **The ratios are clamped; the posterior is not.** Clamping the posterior to $[10^{-4}, 1 - 10^{-4}]$ while leaving ratios free hides an unbounded odds product behind a number that looks like a probability. With $\lambda \in [0.05, 20]$ and $\pi \in (0, 1)$ the posterior lies strictly inside $(0, 1)$ with no clamp at all. The bound is also a statement about sample size: four cells fitted from 46 cases can produce a ratio of 30 from a cell observed three times, and $\lambda_{\max}$ says no witness earns that on this much data.

A unit in which nothing was detected produces evidence rows and *no* fused result; no synthetic “all agents abstained” key is minted into the key space.

Algorithm 4 passes the *whole unit’s* evidence to every key. A silence is a statement about the unit rather than about a finding and therefore carries no key; passing only the keyed group is precisely how silences and abstentions fall out of the arithmetic unnoticed.

VII. CALIBRATION

A. What is estimated

The fitted quantity is a class-conditional likelihood ratio,

$$\Lambda_W(z) = \frac{P(\text{cell}(W) = z \mid y = 1)}{P(\text{cell}(W) = z \mid y = 0)}, \quad (23)$$

Algorithm 4 FUSE($u, \mathcal{E}$)

Input: all evidence $\mathcal{E}$ for unit u ; artifact $\langle \pi, \theta^*, \Lambda \rangle$

Output: one result per detected finding, most probable first

```

1: for all  $e \in \mathcal{E}$  do
2:    $\omega(a(e))$  ▷ an unregistered agent raises here
3: end for
4:  $K \leftarrow \{k(e) : e \in \mathcal{E}, \text{kind}(e) = \text{DETECTION}\}$ 
5: for all  $\kappa \in K$  do
6:    $c \leftarrow$  the unique CWE at  $\kappa$ 
7:   for all  $W \in \mathcal{W}$  do
8:      $z_W \leftarrow \text{cell}(W, \kappa)$  by (19) over all of  $\mathcal{E}$ 
9:      $\lambda_W \leftarrow$  (21); record  $\langle W, z_W, \lambda_W, \text{agents} \rangle$ 
10:   end for
11:    $p \leftarrow$  (22) from  $\{z_W\}$ ,  $\Lambda$  and  $\pi$ 
12: end for
13: return results sorted by  $p$ , descending

```

with both terms conditioned on the witness having said anything at all. Abstentions are excluded from both denominators rather than smoothed into them: including them would make the ratio depend on how often a witness happened to be unavailable, which is a fact about the machine and not about the code.

Equation (23) is also what makes a deliberately balanced corpus legitimate. Both terms are conditioned on the label, so neither depends on prevalence; prevalence enters exactly once, through the prior. A fit that estimated $P(y = 1 \mid z)$ directly would bake the corpus’s 50% into every posterior with no honest way to remove it.

B. Estimator

With Laplace smoothing $\alpha = 1$ over the $|\mathcal{Z}| = 4$ cells, and $n_1(z), n_0(z)$ the counts among claims where W spoke,

$$\hat{p}_y(z) = \frac{n_y(z) + \alpha}{N_y + \alpha |\mathcal{Z}|}, \quad y \in \{0, 1\}, \quad (24)$$

$$\hat{\Lambda}_W(z) = \begin{cases} 1 & n_1(z) = n_0(z) = 0 \\ \hat{p}_1(z) / \hat{p}_0(z) & \text{otherwise,} \end{cases} \quad (25)$$

followed by the clamps $\Lambda_W(S) \leq 0.999$ and $\Lambda_W(z) \in [0.05, 20]$, with the raw value recorded alongside the clamped one so that a bound which bit is visible in the artifact rather than absorbed into the value.

The first branch of (25) is not an omission. Smoothing alone cannot express “no observation”: with both counts zero the smoothed ratio reduces to the class balance of the witness’s *other* cells. Concretely, the runtime witness spoke on 9 vulnerable and 5 safe claims, so its two never-observed detection tiers smoothed to 0.69 — mild evidence of *safety* on tiers that nothing has ever selected. No observation is no evidence, and no evidence is exactly 1.

The silence ceiling is likewise not cosmetic. A silence at or above 1 would make looking-and-finding-nothing into evidence *for* a vulnerability. A witness can nonetheless fit above 1 — which happens when it stays silent on vulnerable

cases at least as often as on safe ones — and that says the witness is weak, not that its silence is incriminating.

C. The prior is declared, not measured

The corpus is twin-paired: every vulnerable case has a safe twin sharing its finding key, so prevalence is 0.5 by construction. That is a fact about how the cases were written, not about how often a changed function is vulnerable, and reading it as a prior would assert that one change in two is vulnerable. The base rate is therefore *declared*, $\pi = 0.03$, and recorded as declared, together with the observed corpus prevalence π_c and the rescaling constant

$$\Xi = \frac{\pi/(1-\pi)}{\pi_c/(1-\pi_c)}, \quad (26)$$

so a published posterior can be converted back to the balanced-corpus figure and checked. Because (23) is prevalence-invariant, changing π rescales every posterior by Ξ and refits no ratio.

D. Importance weighting

A rate measured on a balanced split describes a population that does not exist. With split prevalence π_s , claims are reweighted to the declared base rate by

$$w_+ = \frac{\pi}{\pi_s}, \quad w_- = \frac{1-\pi}{1-\pi_s}, \quad (27)$$

and (27) is applied before any precision, ECE or Brier figure is computed.

E. Threshold selection

Candidate thresholds are the observed posteriors themselves, plus a floor of 0.01, since thresholds between two adjacent posteriors are behaviourally identical. With weighted counts $\text{TP}(\theta) = \sum_{i: p_i \geq \theta} w_i y_i$ and FP, FN defined analogously,

$$\theta^* = \max \left(\arg \max_{\theta} \frac{2 \text{Pr}(\theta) \text{Rc}(\theta)}{\text{Pr}(\theta) + \text{Rc}(\theta)} \right). \quad (28)$$

F_1 balances the two failure modes this project cares about in opposite directions — a missed vulnerability, and an alert a developer learns to ignore — and it needs no second hand-set constant of the kind a precision floor would introduce. Ties break *upward*: two thresholds scoring identically on fourteen validation cases are not really tied, and the higher one alerts on strictly fewer things.

F. Calibration metrics

These are acceptance criteria, not an appendix. With B equal-width bins over $[0, 1]$ and $W_b = \sum_{i \in b} w_i$,

$$\text{ECE} = \frac{1}{\sum_b W_b} \sum_{b: W_b > 0} W_b |\bar{p}_b - \bar{y}_b|, \quad (29)$$

$$\text{Brier} = \frac{\sum_i w_i (p_i - y_i)^2}{\sum_i w_i}, \quad (30)$$

where $\bar{p}_b$ and $\bar{y}_b$ are the weighted mean confidence and observed frequency in bin b . Empty bins contribute nothing

TABLE I
FITTED LIKELIHOOD RATIOS PER WITNESS PER CELL, WITH OBSERVATION COUNTS (n_1/n_0). CALIBRATION SPLIT, $\alpha = 1$.

Witness	H	M	L	S
structural	2.22 ^(1/0)	15.56 ^(13/0)	1.00 ^(0/0)	0.065 ^(0/16)
semantic	9.48 ^(16/1)	0.56 ^(0/1)	0.74 ^(1/2)	0.304 ^(5/21)
context	2.00 ^(1/0)	4.00 ^(3/0)	2.00 ^(1/0)	0.167 ^(0/5)
runtime	6.92 ^(9/0)	1.00 ^(0/0)	1.00 ^(0/0)	0.115 ^(0/5)

TABLE II
PRIOR, THRESHOLD AND CALIBRATION METRICS.

Quantity	Value
Base rate π	0.03, declared (corpus prevalence 0.469)
Threshold θ^*	0.2267, max weighted F_1 on validation
Selection scores	$F_1 = 0.923$, $\text{Pr} = 1.000$, $\text{Rc} = 0.857$; 6 of 15 claims alert
Validation	ECE 0.027, Brier 0.013 (weighted)
Calibration, in-sample	ECE 0.009, Brier 0.010 (weighted)

rather than contributing zero error; averaging their absence in would report a calibration error that falls as the sample shrinks. Brier rewards being right and confident; ECE is the one that speaks to the thesis, since a model can score excellently on (30) and still say 90% when it means 60% — which is precisely the system this project exists to criticise.

G. Split discipline

Ratios (25) are fitted on the calibration split only; the threshold (28) is selected on the validation split only; the test split is evaluated exactly once, at the end. Splits are assigned by twin-pair identifier, never by case identifier, so a twin cannot straddle a split. The fitting routine refuses an observation set from the wrong split, and the observation harness cannot produce a test-split set at all.

VIII. FITTED VALUES

Table I gives the artifact fitted on the calibration split (49 claims from 46 cases); Table II gives the selection and metrics on the validation split (15 claims from 14 cases). Every ratio ships with its per-cell counts and with the corpus and split hashes it was measured against, and there is no hand-set fallback anywhere in the system: a missing artifact raises rather than quietly producing a number nobody measured.

Three entries in Table I must be read with their counts rather than as four confident rows, and all three are recorded in the artifact rather than smoothed away.

- $\Lambda_{\text{str}}(M) = 15.56$ exceeds $\Lambda_{\text{str}}(H) = 2.22$. This is not a claim that a confident taint path is weaker evidence. Thirteen of the taint engine’s fourteen true detections score in the medium band and exactly one scores high, so the high cell is nearly all smoothing prior. It is a consequence of (6), and it is a scoring-calibration question for the structural agent rather than a fusion one.

- The context witness fits from 10 claims and the runtime witness from 14. Both abstain on most units *by construction* — a repository with no relevant history, a function that will not run in a sandbox — so their ratios rest on the fewest observations.
- Cells with no observations are exactly 1.00, by the first branch of (25).

IX. THREATS TO VALIDITY

One-backend structural witness. The Semgrep backend has no build for the platform the observation run was performed on, so `structural.semgrep` abstained on every unit and the structural row of Table I describes the taint engine alone. This is a correctly handled abstention rather than a defect — it costs the posterior exactly 1 — but the ratios must be re-observed on a platform carrying both backends before any structural number is claimed as a property of the witness.

Declared prior. $\pi = 0.03$ is an assumption, stated as one. It rescales every posterior by Ξ and refits nothing, so it can be replaced without re-running an agent.

Corpus scale. 76 hand-written units in 38 twin pairs, split by pair. Small enough that $\lambda_{\max}$ binds for the reason given above; the figures in Table II are selection-time figures on the validation split, and the honest calibration figure is the sealed test split’s.

Unmeasured operational properties. Per-unit p95 latency and the behaviour of the fuel budget on large functions require dedicated hardware rather than a corpus, and are not reported here.

X. CONCLUSION

The arithmetic above is deliberately modest: an odds product with one factor per witness, four class-conditional ratio tables estimated with Laplace smoothing, and a threshold chosen on data that chose nothing else. What makes it a calibration system rather than a scoring heuristic is the discipline around it — a shared finding key so that two witnesses can update one belief, a three-valued evidence algebra so that failing to look costs nothing, a single implementation of “what did this witness say” read by both the fitting and the fusing halves, and a fitted artifact that is the only source of any number in the pipeline.